

Laboratory Work #10

Please write SQL queries for the following tasks and save them as a `lab10.sql` file.

Database Structure: Movie Rental System

Tables:

1. Movies

- movie_id (Primary Key, Integer)
- title (Varchar)
- genre (Varchar)
- price_per_day (Decimal)
- available_copies (Integer)

2. Rentals

- rental_id (Primary Key, Integer)
- movie_id (Foreign Key, Integer)
- customer_id (Integer)
- rental_date (Date)
- quantity (Integer)

3. Customers

- customer_id (Primary Key, Integer)
- name (Varchar)
- email (Varchar)

Sample Data:

Movies

movie_id	title	genre	price_per_day	available_copies
1	The Matrix	Sci-Fi	5.00	8
2	Titanic	Romance	3.50	12
3	Avengers: Endgame	Action	6.00	5

Rentals

rental_id	movie_id	customer_id	rental_date	quantity
1	1	201	01/11/24	2
2	2	202	03/11/24	1
3	3	201	05/11/24	3

Customers

customer_id	name	email
201	Alice Johnson	alice.j@example.com
202	Bob Smith	bob.smith@example.com

Tasks:

1. Transaction for Renting a Movie

- **Scenario:** A customer rents a movie. This should decrease the available copies of the movie in the Movies table and create a new record in the Rentals table.
- **Task:**
 - Start a transaction.
 - Insert a rental record into the Rentals table for customer_id 201 renting 2 copies of movie_id 1.
 - Update the Movies table to decrease the available_copies of movie_id 1 by 2.
 - Commit the transaction.
 - **Expected Outcome:** The Rentals table contains a new record, and the Movies table shows 6 as the new available_copies for movie_id 1.

2. Transaction with Rollback

- **Scenario:** A customer attempts to rent more movies than are available, which should not be allowed.
- **Task:**

- Start a transaction.
- Attempt to insert a rental record into the Rentals table for customer_id 202 renting 10 copies of movie_id 3.
- Check if the available_copies in the Movies table is sufficient. If not, rollback the transaction.
- **Expected Outcome:** No changes are made to the Rentals or Movies tables due to the rollback.

3. Demonstration of Isolation Levels

- **Scenario:** Show the effects of different isolation levels in concurrent transactions.
- **Task:**
 - In one session, start a transaction with the READ COMMITTED isolation level and update the price_per_day of a movie in the Movies table.
 - In a second session, start another transaction at the same isolation level and read the price_per_day of the same movie.
 - Commit the first transaction and re-read the price_per_day in the second session.
 - **Expected Outcome:** The second session reads the updated price after the first transaction commits, demonstrating the READ COMMITTED isolation level.

4. Durability Check

- **Scenario:** Ensure that changes made by a transaction are permanent.
- **Task:**
 - Perform a transaction to update a customer's email in the Customers table.
 - Commit the transaction.
 - Restart the database server.
 - Check the Customers table for the update.

- **Expected Outcome:** The new email persists even after the database restart, demonstrating durability.